Dive into Deep Learning (D2L)

Chapter 3: Linear Neural Networks for Regression

Joon Soo Yoo, Ph.D.

Spring 2026

Assistant Professor
Defense & Advanced Security (DAS) Lab
Jeonbuk National University (JBNU)

Section 3.3

Synthetic Regression Data

인공 데이터 생성을 통한 학습 알고리즘 검증

1. Why Synthetic Data? (가짜 데이터의 가치)

- **목적**: 우리가 설계한 알고리즘이 예상대로 동작하는지 확인하는 **'Sandbox'** 역할
- **장점**:
  - **Ground Truth**: 정답(진짜 파라미터)을 이미 알고 있음 → 모델 성능 평가의 기준 명확
  - **노이즈 제어**: 데이터의 오염 정도(ϵ)를 직접 조절하며 알고리즘의 견고함 테스트 가능

Didactic Purpose

정답을 모르는 문제를 풀기 전에, 정답을 아는 문제부터 제대로 푸는지 확인하는 과정입니다.

1.(1) Workflow: 인공 데이터 생성의 4단계

1. **Step 1. 정답 설정 (Ground Truth)**: 모델이 찾아내야 할 진짜 가중치($\mathbf{w}$)와 편향(b)을 우리가 먼저 결정합니다.
2. **Step 2. 특징 생성 (Features)**: 입력 데이터 $\mathbf{X} \in \mathbb{R}^{n \times d}$를 무작위(Standard Normal)로 생성합니다.
3. **Step 3. 라벨 합성 (Labels with Noise)**: 선형 결합에 노이즈(ϵ)를 추가하여 실제 데이터와 유사한(Realistic) 라벨 $\mathbf{y}$를 만듭니다.
4. **Step 4. 미션 부여 (Objective)**: 모델에게는 $(\mathbf{X}, \mathbf{y})$만 주고, 우리가 숨겨둔 $(\mathbf{w}, b)$를 역으로 추적하게 합니다.

1.(2) Mathematical Goal: 수식으로 보는 생성 원리

데이터 생성 방정식 (Generative Model)

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b + \epsilon$$

- $\mathbf{X} \in \mathbb{R}^{n \times d}$: n개의 샘플, d개의 특징을 가진 행렬
- $\mathbf{w} \in \mathbb{R}^{d \times 1}$: 각 특징에 대응하는 가중치 열 벡터
- $b \in \mathbb{R}$: 스칼라 편향 (Broadcasting을 통해 모든 행에 더해짐)
- $\epsilon \in \mathbb{R}^{n \times 1}$: 관측 오차(Noise), 주로 $N(0, \sigma^2)$ 분포를 따름

우리의 최종 목표 (Inverse Problem)

주어진 $(\mathbf{X}, \mathbf{y})$로부터 $\mathbf{y} \approx \mathbf{X}\hat{\mathbf{w}} + \hat{b}$를 만족하는 최적의 예측값 $\hat{\mathbf{w}}$와 $\hat{b}$를 찾아내는 것!

2. Mathematical Model: 데이터 생성 원리

- 우리는 2차원 특징(Feature)을 가진 1,000개의 예제를 생성합니다.

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b + \epsilon$$

Model Components

성분	설명
$\mathbf{X} \in \mathbb{R}^{1000 \times 2}$	표준 정규 분포에서 추출한 특징 행렬 (Design Matrix)
$\mathbf{w} = [2, -3.4]^\top$	**Ground Truth Weight** (우리가 찾아내야 할 정답 1)
$b = 4.2$	**Ground Truth Bias** (우리가 찾아내야 할 정답 2)
ϵ	노이즈 (평균 0, 표준편차 0.01인 정규분포)

3. Object-Oriented Design: DataModule의 상속

- **상속 구조**: `d2l.DataModule`을 부모 클래스로 사용
- **전략**:
  1. `__init__`에서 모든 데이터 생성 로직을 처리
  2. `save_hyperparameters()`를 호출하여 설정값($\mathbf{w}$, b, 노이즈 등)을 자동 저장
  3. 추후 배치 크기(`batch_size`)를 결정하여 데이터를 공급할 준비를 함

4. 구현: SyntheticRegressionData (Part 1)

Python Code: 클래스 정의 및 속성 저장

```python
class SyntheticRegressionData(d2l.DataModule):  #@save
    """ 선형 회귀를 위한 인공 데이터 생성 클래스 """
    def __init__(self, w, b, noise=0.01, num_train=1000, num_val=1000,
                 batch_size=32):
        super().__init__()
        self.save_hyperparameters()  # w, b, noise 등을 self에 자동 저장

        n = num_train + num_val
        self.X = torch.randn(n, len(w)) # 표준 정규 분포 특징 생성
```

5. 구현: SyntheticRegressionData (Part 2)

Python Code: y

```
# 노이즈 생성
noise = torch.randn(n, 1) * noise

# 행렬 곱 연산: y = Xw + b + noise
# w.reshape((-1, 1))을 통해 행렬 곱 규격(dim)을 맞춤
self.y = torch.matmul(self.X, w.reshape((-1, 1))) + b + noise
```

행렬 곱(matmul) 주의점

X는 (2000×2)이고 **w**는 $(2,)$이므로, 연산을 위해 **w**를 (2×1)로 변형해줘야 행렬 곱 결과가 (2000×1)로 올바르게 나옵니다.

5.(1) reshape(-1, 1)의 마법: 벡터를 열 행렬로

최종 연산 구조

$$\underbrace{\mathbf{X}}_{(2000\times 2)} \times \underbrace{\mathbf{w}}_{(2\times 1)} = \underbrace{\mathbf{y_hat}}_{(2000\times 1)}$$

이제 차원이 일치하므로 `torch.matmul` 연산이 가능해집니다.

6. Ground Truth 설정

- 이제 우리가 원하는 정답 값을 입력하여 데이터 객체를 생성합니다.

Python Code: 데이터 인스턴스 생성

```python
# 정답 파라미터 정의
true_w = torch.tensor([2, -3.4])
true_b = 4.2

# 데이터 모듈 생성
data = SyntheticRegressionData(w=true_w, b=true_b)
```

7. 생성된 데이터 들여다보기

- 생성된 데이터의 첫 번째 샘플을 확인하여 구조를 파악합니다.

Python Code: 첫 번째 데이터 출력

```python
print('features:', data.X[0])
print('label:', data.y[0])
```

Expected Output (예시)

```
features: tensor([ 0.9026,  1.0264])
label: tensor([ 2.5148])
```

8. Dimension Analysis: 데이터의 모양

- 신경망에 데이터를 넣기 전, 차원(Shape) 확인은 필수입니다.

Data Shapes

변수	Shape	의미
`data.X`	(2000, 2)	2,000개의 샘플 (Train 1000 + Val 1000), 각 2개 특징
`data.y`	(2000, 1)	각 샘플에 대한 1개의 스칼라 정답 값

- 학습 시에는 이 중 일부인 32개(`batch_size`)씩 모델에 입력됩니다.

[실습 1] SyntheticRegressionData 클래스 완성

- 아래 코드를 타이핑하여 데이터를 생성하는 "공장"을 만드세요.

Python Code: 클래스 구현

```
!pip install d2l --no-deps

import torch
from d2l import torch as d2l

class SyntheticRegressionData(d2l.DataModule):
    def __init__(self, w, b, noise=0.01, num_train=1000, num_val=1000,
    batch_size=32):
        super().__init__()
        self.save_hyperparameters()
        n = num_train + num_val
        self.X = torch.randn(n, len(w))
        noise = torch.randn(n, 1) * noise
        # y = Xw + b + noise 구현
        self.y = torch.matmul(self.X, w.reshape((-1, 1))) + b + noise
```

[실습 2] 데이터 생성 및 샘플 확인

- 정답($w = [2, -3.4], b = 4.2$)을 설정하고 데이터를 생성해 봅시다.

Python Code: 인스턴스화 및 출력

```python
# 1. 데이터 객체 생성
true_w = torch.tensor([2, -3.4])
true_b = 4.2
data = SyntheticRegressionData(w=true_w, b=true_b)

# 2. 첫 번째 샘플 출력 특징과( 라벨 확인)
print('X[0]:', data.X[0])
print('y[0]:', data.y[0])

# 3. 전체 데이터 차원 확인
print('X shape:', data.X.shape)
print('y shape:', data.y.shape)
```

[실습 3] 시각화 (Visualization)

- 생성된 데이터가 선형적인 관계를 갖는지 산점도(Scatter Plot)로 확인합니다.

Python Code: 시각화 라이브러리 활용

```python
import matplotlib.pyplot as plt

# 첫 번째 특징(X[:, 0])과 정답(y)의 관계 시각화
plt.figure(figsize=(5, 3))
plt.scatter(data.X[:, 0].detach().numpy(),
            data.y.detach().numpy(), s=1);
plt.xlabel('x1'); plt.ylabel('y');
plt.title('Relationship between x1 and y')
plt.show()
```

실습 정리 및 생각해볼 점

Check-list

- `data.X`의 열(Column) 개수가 2개인 것을 확인했나요?
- `noise` 값을 0.1로 키우면 산점도가 어떻게 변하나요?

Section 3.4

Linear Regression from Scratch

바닥부터 조립하며 배우는 선형 회귀의 원리

1. Why Implementation from Scratch?

- **목표**: 프레임워크의 자동화 기능 없이 텐서와 미분만으로 모델을 완성함
- **이득**: 딥러닝 내부의 수학적 동작 원리를 코드로 완벽히 체득 가능

2. 모델의 뼈대와 파라미터 초기화

- 학습을 시작하기 위해 가중치($\mathbf{w}$)와 편향(b)의 시작 지점을 정합니다.

Python Code: __init__ 구현

```python
class LinearRegressionScratch(d2l.Module): #@save
    # 바닥부터 구현하는 선형 회귀 모델
    def __init__(self, num_inputs, lr, sigma=0.01):
        super().__init__()
        self.save_hyperparameters()
        # 가중치: 입력( 개수 x 1) 형태의 정규분포 랜덤값
        self.w = torch.normal(0, sigma, (num_inputs, 1), requires_grad=True)
        # 편향: 으로0 시작
        self.b = torch.zeros(1, requires_grad=True)
```

3.(1) 파라미터 초기화: 랜덤한 첫걸음

- `torch.normal(0, sigma, size)`:
  - 평균 0, 표준편차 `sigma`인 정규분포($N(0, \sigma^2)$)에서 숫자를 무작위로 추출함
  - `(num_inputs, 1)`: 입력 개수만큼의 행과 1개의 열을 가진 '열 벡터'를 만듦
- `torch.zeros(1)`: 편향(b)을 일단 0으로 설정하여 깨끗한 상태에서 시작함

3.(2) 학습의 핵심: `requires_grad = True`

- **Autograd (자동 미분)**: 파이토치가 해당 텐서에 일어나는 모든 연산을 가계부처럼 기록하기 시작함
- **목적**: 나중에 오차(Loss)가 발생했을 때, "어떤 가중치를 얼마나 고쳐야 오차가 줄어들지" 계산하기 위해 미분값이 필요하기 때문임

Grad 옵션 유무의 차이

설정	상태
`False` (기본값)	일반적인 숫자 덩어리. 값이 변하지 않음.
`True`	**학습 가능한 파라미터**. 미분을 통해 스스로 진화할 준비가 됨.

4. Forward Pass: 어떻게 예측하는가?

- 입력 데이터 $\mathbf{X}$와 가중치 $\mathbf{w}$의 행렬 곱에 편향 b를 더함

Mathematical Formulation

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

5. 구현: `forward` 메서드

- `Module` 클래스의 인터페이스에 맞춰 예측 연산을 구현합니다.

Python Code: forward 연산

```python
@d2l.add_to_class(LinearRegressionScratch) #@save
def forward(self, X):
    # 예측값 산출: y_hat = Xw + b
    return torch.matmul(X, self.w) + self.b
```

- `torch.matmul`을 사용해 행렬 연산을 효율적으로 처리합니다.

Loss Function & Optimization

1. 손실 함수

- **Squared Loss**: 예측값($\hat{y}$)과 실제 정답(y)의 차이를 제곱하여 측정함

Mathematical Formula

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

2. 구현: loss 메서드

Python Code: Squared Loss

```python
@d2l.add_to_class(LinearRegressionScratch) #@save
def loss(self, y_hat, y):
    # 예측값과 정답의 차이를 제곱하여 평균을 냄
    l = (y_hat - y) ** 2 / 2
    return l.mean()
```

- `mean()`: 미니배치 전체의 오차를 하나의 숫자로 평균 내어 최적화하기 쉽게 만듦

2.(1)손실 함수의 벡터 연산 과정

- **벡터화 연산(Vectorized Operation)**: 모든 샘플의 오차를 동시에 계산함

1단계: 요소별 오차(Element-wise Loss) 계산

$$\mathbf{l} = \begin{bmatrix} l^{(1)} \\ \vdots \\ l^{(32)} \end{bmatrix} = \frac{1}{2}(\hat{\mathbf{y}} - \mathbf{y})^2 = \frac{1}{2} \begin{bmatrix} (\hat{y}^{(1)} - y^{(1)})^2 \\ \vdots \\ (\hat{y}^{(32)} - y^{(32)})^2 \end{bmatrix}$$

2단계: 최종 손실값(Scalar Loss) 산출

$$L = \text{mean}(\mathbf{l}) = \frac{1}{n} \sum_{i=1}^{n} l^{(i)}$$

3. 최적화: 소량의 데이터로 길 찾기

- **Minibatch SGD**: 전체 데이터가 아닌 일부(Batch)만 보고 경사를 계산함
- **Update Rule**: 경사(Gradient)의 반대 방향으로 아주 조금씩 이동함

4. 구현: class SGD

Python Code: SGD Optimizer

```python
class SGD(d2l.HyperParameters): #@save
    # 미니배치 확률적 경사 하강법
    def __init__(self, params, lr):
        self.save_hyperparameters()

    def step(self, _):
        # 파라미터를 경사 방향의 반대로 업데이트
        for param in self.params:
            param -= self.lr * param.grad
```

- `param.grad`: 앞서 계산된 미분값(경사)
- `lr`: 보폭(Learning Rate). 너무 크면 골짜기를 지나치고, 작으면 너무 느림

4.(1) SGD 업데이트: 경사를 따라 내려가기

- **업데이트 규칙**: 현재 위치($\mathbf{w}$)에서 경사의 반대 방향으로 이동함

Mathematical Formula

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} L$$

수식 기호 설명

기호	코드	의미
$\mathbf{w}$	`param`	우리가 최적화하려는 가중치 (현재 위치)
η	`self.lr`	학습률 (보폭, Learning Rate)
$\nabla_{\mathbf{w}} L$	`param.grad`	손실함수(L)를 가중치($\mathbf{w}$)로 **편미분**한 값

4.(2) 자동 미분(Autograd): 누가 편미분을 하나?

- **누가 계산하나?**: 우리가 `loss.backward()`를 호출하는 순간, 파이토치의 엔진이 역전파(Backpropagation)를 통해 모든 `param`의 편미분 값을 계산함
- **무엇을 계산하나?**: 현재의 **w**와 b 값에서의 **국소적 기울기(Partial Derivative)**를 계산함

편미분의 실체

$$\text{param.grad} = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \ldots, \frac{\partial L}{\partial b}\right]_{\text{at current point}}$$

5. 연결: configure_optimizers

- 모델에게 "너를 가르칠 선생(Optimizer)은 이 녀석이야"라고 알려주는 단계입니다.

Python Code: Optimizer 설정

```python
@d2l.add_to_class(LinearRegressionScratch) #@save
def configure_optimizers(self):
    # 학습할 대상(w, b)과 학습률(lr)을 전달함
    return SGD([self.w, self.b], self.lr)
```

[실습 4] 모델 정의 및 초기화

- w와 b를 직접 생성하고, 미분 가능하도록 설정(requires_grad)합니다.

Python Code: 모델 클래스 구현

```python
class LinearRegressionScratch(d2l.Module):
    def __init__(self, num_inputs, lr, sigma=0.01):
        super().__init__()
        self.save_hyperparameters()
        # 가중치 (num_inputs x 1)와 편향 (1) 초기화
        self.w = torch.normal(0, sigma, (num_inputs, 1), requires_grad=True)
        self.b = torch.zeros(1, requires_grad=True)

    def forward(self, X):
        # 예측 연산: y = Xw + b
        return torch.matmul(X, self.w) + self.b
```

[실습 5] 손실 함수 구현

- 예측값과 정답 사이의 거리를 측정하는 수식을 추가합니다.

Python Code: Loss 메서드 추가

```python
@d2l.add_to_class(LinearRegressionScratch)
def loss(self, y_hat, y):
    # Squared Error: (y_hat - y)^2 / 2
    l = (y_hat - y)**2 / 2
    return l.mean()
```

[실습 6] 최적화 알고리즘 (SGD) 구현

- 계산된 경사(Gradient)를 따라 파라미터를 업데이트하는 클래스를 만듭니다.

Python Code: SGD Optimizer

```python
class SGD(d2l.HyperParameters):
    def __init__(self, params, lr):
        self.save_hyperparameters()

    @d2l.add_to_class(SGD)
    def step(self):
        # 가중치 업데이트 시 미분 기록을 일시적으로 끕니다.
        with torch.no_grad():
            for param in self.params:
                param -= self.lr * param.grad
                # 다음 배치를 위해 기울기 값을 으로0 지워줍니다.
                param.grad.zero_()
```

[실습 7] 모델과 최적화 도구 연결

- 모델이 자신의 가중치를 업데이트할 때 어떤 도구를 쓸지 결정합니다.

Python Code: configure_optimizers

```python
@d2l.add_to_class(LinearRegressionScratch)
def configure_optimizers(self):
    # 학습 대상([w, b])과 학습률(lr)을 전달
    return SGD([self.w, self.b], self.lr)
```

[실습 7.5] 데이터 공급 장치(DataLoader) 구현

Python Code: get_dataloader (제너레이터 방식)

```python
import random
import numpy as np

@d2l.add_to_class(SyntheticRegressionData)
def get_dataloader(self, train):
    if train:
        indices = list(range(0, self.num_train))
        # 1. 학습 데이터를 무작위로 섞음
        random.shuffle(indices)
    else:
        indices = list(range(self.num_train, self.num_train + self.num_val))

    # 2. 배치 크기만큼 데이터를 슬라이싱하여 반환
    for i in range(0, len(indices), self.batch_size):
        batch_indices = np.array(indices[i: i + self.batch_size])
        yield self.X[batch_indices], self.y[batch_indices]
```

[실습 7.5] 데이터 공급 장치(DataLoader) 구현

- **역할**: 전체 데이터를 무작위로 섞고, 설정한 배치 크기(32)만큼 잘라서 모델에 전달합니다.
- **무작위 추출(Shuffle)**: 매 에폭마다 데이터 순서를 섞어 모델이 순서를 외우는 것을 방지합니다.

Yield 키워드의 의미

데이터 전체를 메모리에 올리지 않고, 필요할 때마다 한 배치씩 "생성해서(Yield)" 던져주는 효율적인 방식임.

[실습 8] 중간 점검: Forward & Loss Test

- 학습 루프를 돌리기 전, 데이터가 모델을 한 번 통과해 보는지 확인합니다.

Python Code: Sanity Check

```python
# 1. 모델 객체 생성 특징( 개수 개2, 학습률 0.03)
model = LinearRegressionScratch(num_inputs=2, lr=0.03)

# 2. 데이터 배치를 하나 꺼내서 예측 수행
X, y = next(iter(data.train_dataloader()))
y_hat = model(X)

# 3. 오차 계산
current_loss = model.loss(y_hat, y)

print('y_hat shape:', y_hat.shape)
print('Current Loss:', current_loss)
```

Training: 모델을 실제로 가르치기

모든 부품을 하나로 연결하는 핵심 알고리즘

1. 학습 알고리즘의 4단계

- **반복(Epoch)**: 데이터셋 전체를 한 번 다 훑을 때까지 다음을 반복함

수학적 워크플로우

1. **Initialize**: 파라미터 $(\mathbf{w}, b)$를 무작위로 초기화
2. **Compute Gradient**: 미니배치($\mathcal{B}$)의 평균 경사를 계산

$$\mathbf{g} \leftarrow \partial_{(\mathbf{w},b)} \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} l(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w}, b)$$

3. **Update**: 경사의 반대 방향으로 파라미터 업데이트

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \eta \mathbf{g}$$

2. 구현: prepare_batch

- 로더에서 꺼내온 배치를 모델에 넣기 좋게 다듬는 단계입니다.

Python Code: Trainer 확장

```python
@d2l.add_to_class(d2l.Trainer) #@save
def prepare_batch(self, batch):
    # 현재는 데이터를 그대로 반환함 추후(GPU 이동 등에 활용)
    return batch
```

3. 구현: fit_epoch (가장 중요한 코드)

Python Code: 한 에폭 동안의 학습 과정

```python
@d2l.add_to_class(d2l.Trainer) #@save
def fit_epoch(self):
    # 1. 훈련 데이터 로더에서 배치를 하나씩 꺼냄
    for batch in self.train_dataloader:
        # 2. 오차(Loss) 계산 (Forward Pass)
        loss = self.model.training_step(self.prepare_batch(batch))
        # 3. 경사(Gradient) 계산 (Backward Pass)
        loss.backward()
        # 4. 파라미터 업데이트 (Step)
        self.optim.step()
        self.train_batch_idx += 1
```

3.(1) fit_epoch

- **학습 루프**: 미니배치 데이터를 하나씩 꺼내어 모델을 업데이트하는 과정

Code vs. Math Mapping

코드 한 줄	수학적 동작	의미
`model.training_step`	$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$ 및 $L = \text{loss}(\hat{\mathbf{y}}, \mathbf{y})$	예측 후 오차 측정
`loss.backward()`	$\nabla_{\mathbf{w},b} L = \frac{\partial L}{\partial(\mathbf{w},b)}$	경사(Gradient) 계산
`optim.step()`	$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \eta \nabla L$	실제 가중치 업데이트

3.(2) training_step

- **정체**: 이 함수는 내부적으로 `forward()`와 `loss()`를 동시에 호출함

수식으로 보는 내부 연산

$$L = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2}\left(\underbrace{\mathbf{X}^{(i)}\mathbf{w} + b}_{\text{forward}} - y^{(i)}\right)^2$$

3.(3) `loss.backward()`

- **수학적 실체**: 연쇄 법칙(Chain Rule)을 이용한 편미분 연산
- **결과**: 각 가중치 주머니(`w.grad`, `b.grad`)에 현재 위치의 기울기 값이 채워짐

Mathematical Goal

$$\text{Find } \mathbf{g} = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b} \right]$$

4. 실력 점검: Validation 과정

- 학습 중간중간에 모델이 처음 보는 데이터(Validation Set)를 얼마나 잘 맞히는지 테스트합니다.

Python Code: fit_epoch 내 검증 로직

```python
if self.val_dataloader is None:
    return
# 검증 모드: 파라미터를 업데이트하지 않고 성능만 측정
for batch in self.val_dataloader:
    self.model.validation_step(self.prepare_batch(batch))
    self.val_batch_idx += 1
```

4.(1) Validation Loop: 실력 점검의 시간

- **Safety Guard**: `val_dataloader`가 없으면 검증 단계를 건너뜁니다.
- `validation_step`: 학습용이 아닌 '새로운 데이터'로 오차(L)를 측정합니다.

Train vs. Validation

구분	동작	목적
Training	예측 → 오차 → **업데이트**	실력 향상 (공부하기)
Validation	예측 → 오차	**실력 측정** (모의고사)

Execution and Evaluation

모델 학습 실행 및 파라미터 추정 결과 분석

1. Hyperparameters: 학습의 제어 변수

- **Learning Rate (η)**: 0.03으로 설정 (가중치 업데이트의 보폭)
- **Max Epochs**: 3으로 설정 (전체 데이터셋을 3번 반복 학습)

하이퍼파라미터(Hyperparameter)의 특징

- 모델이 스스로 학습하는 파라미터(w, b)와 달리, **사용자가 직접 결정**해야 하는 변수입니다.
- 설정값에 따라 학습의 성패가 갈리며, 일반적으로 Training / Validation / Test의 3-way split을 통해 최적값을 찾습니다.

2. 모델 학습 실행 (Model Fitting)

- 생성한 인공 데이터와 모델, 트레이너를 결합하여 학습을 시작합니다.

Python Code: Training Process

```
# 1. 모델 인스턴스 생성 입력( 크기 2, 학습률 0.03)
model = LinearRegressionScratch(2, lr=0.03)

# 2. 인공 데이터 생성 정답(: w=[2, -3.4], b=4.2)
data = d2l.SyntheticRegressionData(w=torch.tensor([2, -3.4]), b=4.2)

# 3. 트레이너 설정 및 학습 실행
trainer = d2l.Trainer(max_epochs=3)
trainer.fit(model, data)
```

2.(1) 호출 계층도: `fit` vs `fit_epoch`

- **구조적 흐름**: `trainer.fit`이 전체 에폭 루프를 돌며 `fit_epoch`을 호출함

계층 구조 (Hierarchy)

$$\text{trainer.fit()} \rightarrow \begin{cases} \text{epoch 1:} & \text{fit_epoch()} \rightarrow \text{batch iterations} \\ \text{epoch 2:} & \text{fit_epoch()} \rightarrow \text{batch iterations} \\ \vdots & \\ \text{epoch } N: & \text{fit_epoch()} \rightarrow \text{batch iterations} \end{cases}$$

실행 단계별 역할

단계	주요 동작
시동	`trainer.fit(model, data)` 실행 및 하이퍼파라미터 로드
루프	`max_epochs`만큼 반복 루프(Iteration) 시작
실행	`self.fit_epoch()` 호출 (배치 처리 및 가중치 업데이트)
기록	에폭 종료 후 오차(Loss) 및 성능 지표 기록

2.(2) 설계 철학: 왜 fit은 숨겨져 있는가?

- **관심사의 분리**: "변하지 않는 부분"과 "핵심 학습 로직"을 분리함

추상화(Abstraction) 전략

구분	함수	특징
보일러플레이트	`fit`	에폭 반복, 로그 기록 등 모든 모델에서 공통적인 부분
딥러닝의 심장	`fit_epoch`	배치를 꺼내고, 미분하고, 업데이트하는 핵심 로직

구현의 유연성

- `fit`: 라이브러리 깊숙한 곳에 숨겨두어 코드의 복잡도를 낮춤
- `fit_epoch`: 모델이나 최적화 방식에 따라 달라질 수 있으므로, 사용자(우리)가 직접 코딩하여 원리를 체득하게 함

3. 평가: 추정된 파라미터의 정확도

- 우리가 데이터 생성 시 설정했던 진짜 값($\mathbf{w}, b$)과 모델이 학습한 값($\hat{\mathbf{w}}, \hat{b}$)을 비교합니다.

Python Code: Error Estimation

```python
# 와w 의b 추정 오차 계산
print(f'error in estimating w: {data.w - model.w.reshape(data.w.shape)}')
print(f'error in estimating b: {data.b - model.b}')
```

실험 결과 (Example)

항목	오차값 (Error)
Weight (w)	[0.1108, -0.1269]
Bias (b)	[0.1921]

4. Discussion: 정답을 완벽히 맞힐 수 있는가?

Insight

머신러닝의 궁극적인 목표는 **원래의 파라미터를 찾는 것**이 아니라, **높은 예측 정확도 (Prediction Accuracy)**를 확보하는 것입니다.

[실습 9] Trainer 기능 확장: fit_epoch 구현

- 우리가 만든 `Trainer`에 실제 학습 루프 기능을 주입합니다.

Python Code: fit_epoch 주입

```python
@d2l.add_to_class(d2l.Trainer)
def prepare_batch(self, batch):
    return batch # 현재는 데이터 변환 없이 반환

@d2l.add_to_class(d2l.Trainer)
def fit_epoch(self):
    self.model.train() # 학습 모드로 설정
    for batch in self.train_dataloader:
        # 1. Forward Pass & Loss 계산
        loss = self.model.training_step(self.prepare_batch(batch))
        # 2. Backward Pass (Gradient 계산)
        loss.backward()
        # 3. Parameter Update
        self.optim.step()
        self.train_batch_idx += 1
```

[실습 10] 모델 학습 실행 (Run Training)

- 모든 부품을 연결하여 정답을 향한 3번의 에폭을 수행합니다.

Python Code: Training Process

```python
# 1. 모델과 데이터 준비
model = LinearRegressionScratch(num_inputs=2, lr=0.03)
data = d2l.SyntheticRegressionData(w=torch.tensor([2, -3.4]), b=4.2)

# 2. 트레이너 생성 및 학습 시작
trainer = d2l.Trainer(max_epochs=3)
trainer.fit(model, data)
```

[실습 11] 추정 결과 검증 (Evaluation)

- 우리가 숨겨두었던 정답($w = [2, -3.4], b = 4.2$)과 모델의 결과를 비교합니다.

Python Code: 파라미터 오차 계산

```python
# 모델이 학습한 와w b 꺼내기
learned_w = model.w.data
learned_b = model.b.data

# 실제 정답(true_w, true_b)과의 오차 출력
print(f'Error in w: {data.w - learned_w.reshape(data.w.shape)}')
print(f'Error in b: {data.b - learned_b}')
```

Section 3.5

Concise Implementation

프레임워크 API를 활용한 효율적인 모델 구현

1. Why High-level APIs?

- **생산성**: 매번 바닥부터 구현하는 것은 비효율적임
- **안정성**: 수많은 개발자에 의해 검증된 레이어(Layer), 손실 함수, 최적화 도구를 제공함
- **추상화**: 복잡한 선형 대수 연산을 `nn.Linear`와 같은 클래스 하나로 묶어 관리함

핵심 개념: Fully Connected Layer

각 입력이 가중치 행렬을 통해 모든 출력과 연결되는 구조로, PyTorch에서는 `nn.Linear`, Gluon에서는 `nn.Dense`라고 불립니다.

2. 모델 정의: 단 한 줄로 조립하기

- 우리가 직접 선언했던 $\mathbf{w}, b$가 `nn.Linear`라는 객체 안에 포함됩니다.

Python Code: LinearRegression 구현

```python
class LinearRegression(d2l.Module): #@save
    def __init__(self, lr):
        super().__init__()
        self.save_hyperparameters()
        # 입력 차원은 나중에 데이터가 들어올 때 자동으로 결정됨
        # 출력 차원만 1 로 둠
        self.net = nn.Linear(in_features=None, out_features=1)
        # 가중치 초기화 정규분포()
        self.net.weight.data.normal_(0, 0.01)
        self.net.bias.data.fill_(0)
```

2.(1) normal_ : 왜 언더바(_)가 붙을까?

- **In-place 연산**: 메서드 이름 끝에 _가 붙으면, 새로운 텐서를 만들지 않고 **기존 텐서의 값을 직접 수정**한다는 뜻입니다.

normal vs normal_

함수명	동작 방식	메모리 영향
normal()	새로운 메모리 공간에 결과값을 저장하여 반환	새로운 메모리 할당 필요
normal_()	**현재 위치(In-place)**의 값을 직접 덮어씀	추가 메모리 소모 없음

사용 이유: 효율성(Efficiency)

딥러닝은 거대한 행렬을 다룹니다. 매번 새로운 텐서를 복사해서 만들면 메모리가 금방 고갈되므로, 초기화나 간단한 산술 연산 시에는 _가 붙은 In-place 연산을 사용하여 자원을 절약합니다.

3. 뼈대의 변화: Manual vs. Concise

파라미터 관리의 변화

구분	Section 3.4 (Scratch)	Section 3.5 (Concise)
파라미터 선언	`torch.normal` 직접 선언	`nn.Linear` 내부 변수 활용
미분 설정	`requires_grad=True` 명시	프레임워크가 자동 관리
초기화	직접 변수에 할당	`net.weight.data` 조작

자동 형태 추론 (Automatic Inference)

프레임워크는 첫 번째 데이터 배치가 모델을 통과할 때, 데이터의 크기를 보고 입력 가중치($\mathbf{w}$)의 차원을 자동으로 결정해 줍니다.

4. Forward Pass: 정의된 레이어 호출

- 더 이상 수동으로 `matmul`을 수행할 필요 없이, 레이어 객체를 함수처럼 호출합니다.

Python Code: forward 구현

```python
@d2l.add_to_class(LinearRegression) #@save
def forward(self, X):
    # 내장된 __call__ 메서드를 통해 연산 수행
    # 내부적으로 Xw + b 를 처리함
    return self.net(X)
```

학습 포인트

객체 지향 설계 덕분에 `forward` 메서드는 "데이터가 어떻게 흐르는가"에만 집중할 수 있게 되었습니다.

5. 요약: 조립식 인공지능

- **부품(Layer)**: `nn.Linear`는 선형 연산을 수행하는 완성된 부품입니다.
- **조립(Container)**: `self.net`에 부품을 등록하는 것만으로 모델이 완성됩니다.

Loss Function & Optimizer (Concise)

1. 손실 함수: 바퀴를 새로 만들지 말자

- **MSELoss**: 우리가 직접 짰던 `Squared Loss`를 프레임워크가 제공합니다.
- **특징**: 수치적으로 더 안정적이며, GPU 연산에 최적화되어 있습니다.

Python Code: Loss 구현

```python
@d2l.add_to_class(LinearRegression) #@save
def loss(self, y_hat, y):
    # nn.MSELoss()는 (y_hat - y)의^2 평균을 계산함
    fn = nn.MSELoss()
    return fn(y_hat, y)
```

수학적 일치

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_hat^{(i)} - y^{(i)})^2$$

`nn.MSELoss`는 기본적으로 평균(`mean`) 연산을 포함하고 있습니다.

2. 구현: configure_optimizers

- 모델 내부의 파라미터들을 최적화 도구에 연결해 줍니다.

Python Code: Optimizer 설정

```python
@d2l.add_to_class(LinearRegression) #@save
def configure_optimizers(self):
    # self.parameters()는 모델의 모든 학습 가능한 변수를 반환함
    return torch.optim.SGD(self.parameters(), self.lr)
```

중요: 용어의 혼동 주의

- **D2L Trainer**: 우리가 만든 '학습 루프 전체'를 관리하는 기계
- **Framework Optimizer**: 가중치 업데이트 '공식'만 담당하는 부품

3. 결론: Scratch vs. Concise

구현 방식의 차이

항목	Scratch (3.4)	Concise (3.5)
Loss	`(y_hat - y)**2 / 2`	`nn.MSELoss()`
Optimizer	`param -= lr * grad`	`torch.optim.SGD`
매개변수	`self.w, self.b` 직접 전달	`self.parameters()` 자동 수집

Training and Evaluation

고수준 API를 활용한 통합 학습 실행

1. Advantages of High-level API in Training

- **코드의 간결성**: 파라미터 할당, 손실 함수 정의, SGD 구현 등의 보일러플레이트 코드가 대폭 감소함
- **확장성**: 모델이 복잡해질수록(Layer가 많아질수록) API 활용의 이득은 기하급수적으로 커짐
- **로직의 일관성**: 내부 부품은 바뀌었지만, 우리가 앞서 구축한 `Trainer.fit` 인터페이스는 그대로 유지됨

1.(1) 로직의 일관성: 시동 버튼은 변하지 않는다

- **인터페이스(`Trainer.fit`)**: 모델을 학습시키는 "규격화된 방법"
- **내부 구현**: 3.4절(Scratch)과 3.5절(Concise)의 방식 차이

내부 부품의 교체 (Replacement)

구분	3.4 Scratch (수동)	3.5 Concise (자동)
모델 엔진	우리가 직접 짠 `Xw + b`	`nn.Linear` 부품
최적화 도구	우리가 직접 짠 `SGD`	`torch.optim.SGD` 부품

2. Implementation: 통합 실행

- 3.4절의 Scratch 구현과 동일한 하이퍼파라미터($lr = 0.03$, $max_epochs = 3$)를 사용합니다.

Python Code: Concise Training

```python
# 1. 고수준 API 기반 모델 및 인공 데이터 생성
model = LinearRegression(lr=0.03)
data = d2l.SyntheticRegressionData(w=torch.tensor([2, -3.4]), b=4.2)

# 2. 트레이너 인스턴스 생성 및 학습
trainer = d2l.Trainer(max_epochs=3)
trainer.fit(model, data)
```

- **참고**: `trainer.fit`은 우리가 앞서 정의한 `fit_epoch`의 메커니즘을 그대로 계승하여 작동합니다.

3. 학습된 파라미터 확인

- `nn.Linear` 레이어 내부에 저장된 가중치와 편향 값을 추출합니다.

Python Code: get_w_b 구현

```python
@d2l.add_to_class(LinearRegression) #@save
def get_w_b(self):
    # net.와weight net.에서bias 실제 텐서 데이터를 반환
    return (self.net.weight.data, self.net.bias.data)

w, b = model.get_w_b()
```

- `.data`: 미분 정보를 제외한 순수 수치 데이터에 접근하는 방식입니다.

4. 정답(Ground Truth)과의 비교

- 학습이 완료된 후, 모델이 추정한 값과 실제 생성 데이터의 파라미터를 비교합니다.

추정 정확도 확인 (예시)

$$\text{Error in } \mathbf{w} : \text{data.w} - \text{model.w} \approx [0.0001, -0.0005]$$

$$\text{Error in } b : \text{data.b} - \text{model.b} \approx [0.0002]$$

결과 해석
데이터의 양이 충분하고 모델이 적절히 설정되었다면, Concise 구현 역시 Scratch 구현과 마찬가지로 실제 파라미터에 매우 근접한 결과를 도출합니다.

5. Summary: Concise vs. Scratch

- **핵심 차이**: 구현의 깊이(Abstraction Level). 원리는 동일하지만 도구의 효율성이 극대화됨
- **권장 사항**:
  - 학습 단계: Scratch 구현을 통해 **Deep Understanding** 확보
  - 실무/연구 단계: Concise 구현을 통해 **Fast Prototyping** 및 안정성 확보

Final Insight

고수준 API는 단순히 코드를 줄여주는 것이 아니라, 연구자가 '구현의 디테일'이 아닌 '모델의 구조와 아이디어'에 집중할 수 있게 해주는 강력한 무기입니다.

질의 & 응답

Any Question?